Inside Dark Studio

GitHub Flow para Unity

Una tarea, una rama.
Una versión compartida
que sigue siendo jugable.

KT del equipo
GitHub Desktop + Unity



02

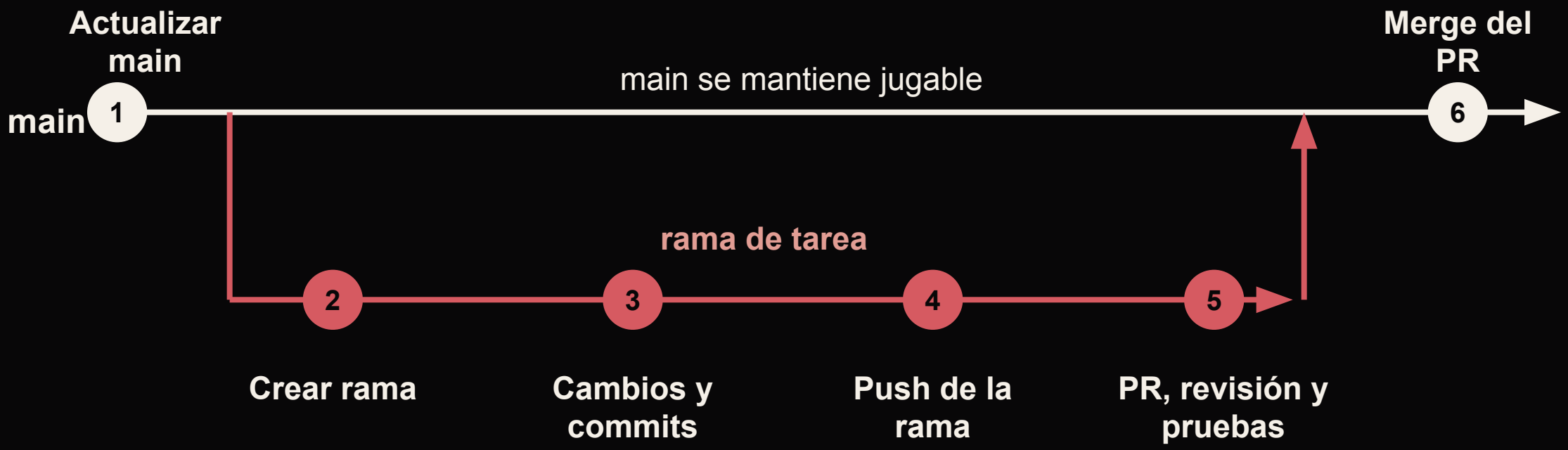
El acuerdo del equipo

main es la versión compartida y jugable



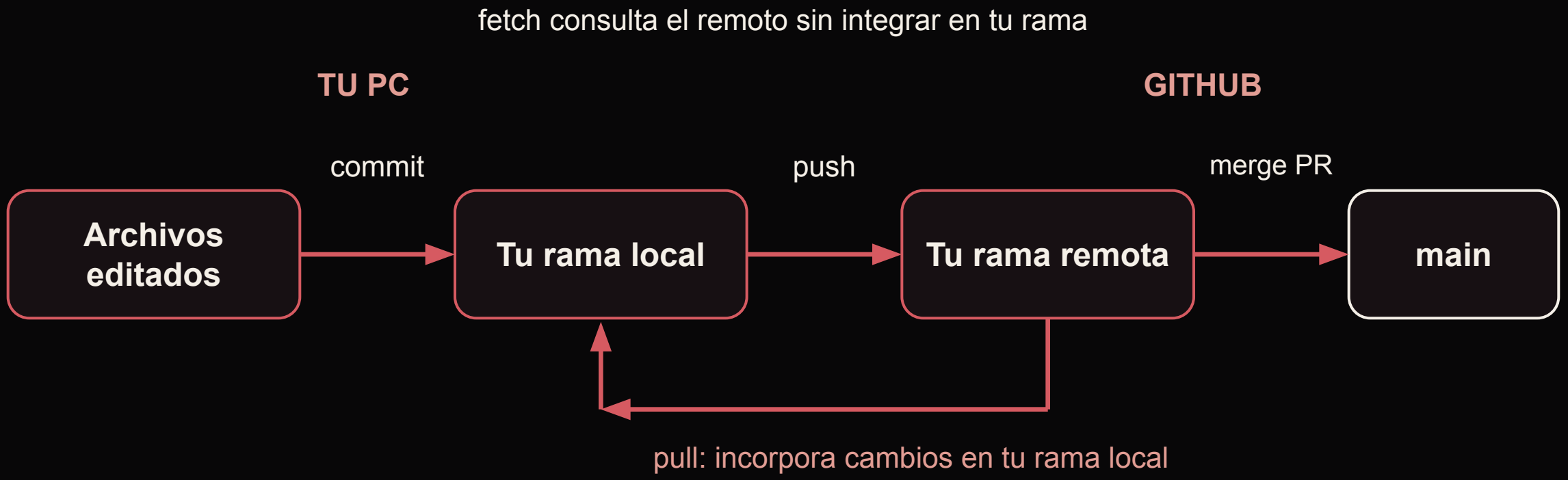
Cada tarea vive en su propia rama.
Integramos cambios pequeños, terminados y probados.
Los experimentos van en una rama aparte.

El recorrido de una tarea



Después del merge: actualizar main local.
La próxima tarea empieza en una rama nueva.

Local y remoto



El merge del PR es el paso que integra en main.

Antes de empezar

Guardar escenas y cambios pendientes. Cerrar Unity.
En Desktop: elegir main y hacer Fetch / Pull origin.
Crear una rama desde ese main actualizado.

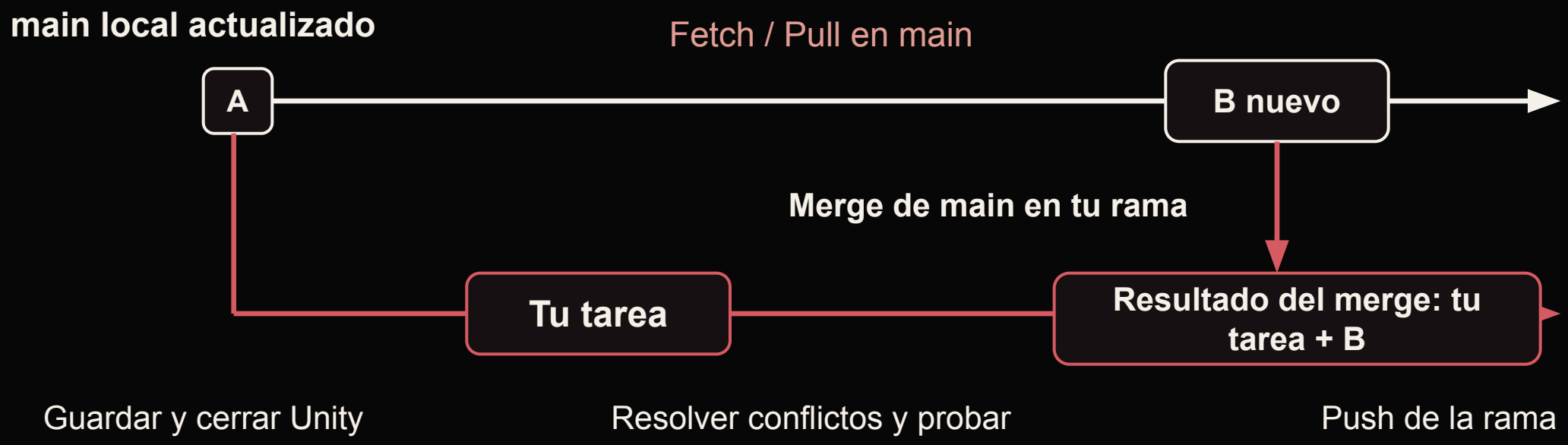
feature/recompensa-moneda
fix/interaccion-npc
o bien usando el id de tarea

Commits que se pueden revisar

Una modificación concreta por commit.
Revisar el diff y cada archivo seleccionado.
Incluir los .meta de los assets nuevos o movidos.
((GUID) +ajustes de importación.)

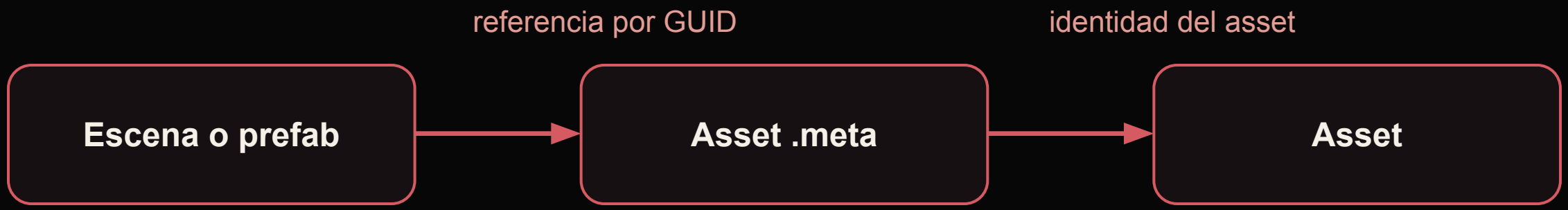
Ejemplo: “Aumenta la recompensa de moneda a 5”

¿Qué hago si main se actualizó mientras yo trabajaba en mi tarea?



Pull de tu rama personal no incorpora main por sí solo.

Unity también versiona referencias



El asset y su .meta se mueven y versionan juntos

También se comparten: Packages y ProjectSettings

.gitignore se versiona; excluye Library/, Temp/ y otros archivos generados.

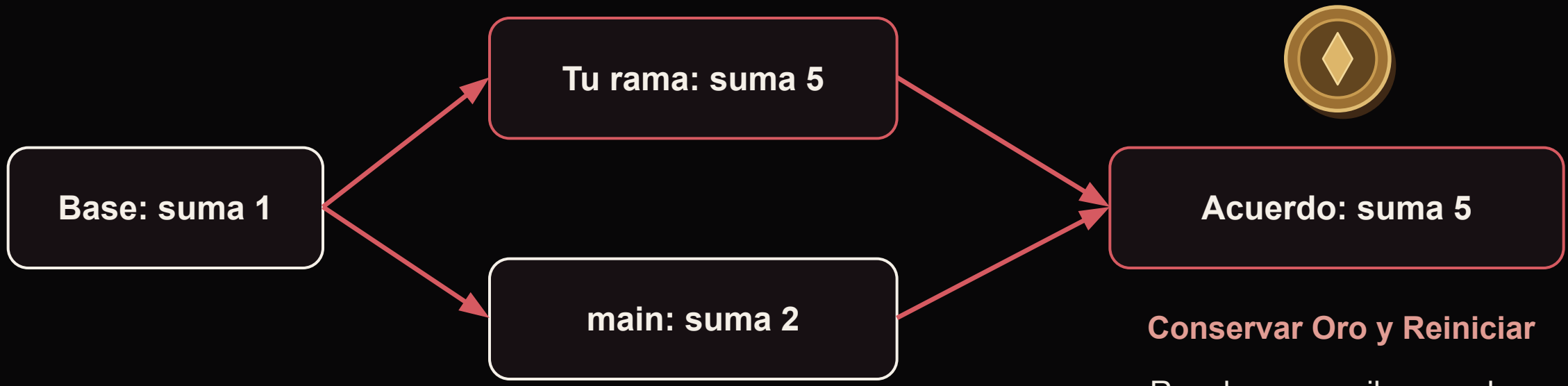
Coordinamos quién edita cada escena, prefab o controller.

```
# Common files to ignore
*.log
node_modules/
.DS_Store
.env
build/
dist/

# But keep this file
!important.log
```

Conflictos con una decisión explícita

Git detecta el conflicto



Conservar Oro y Reiniciar

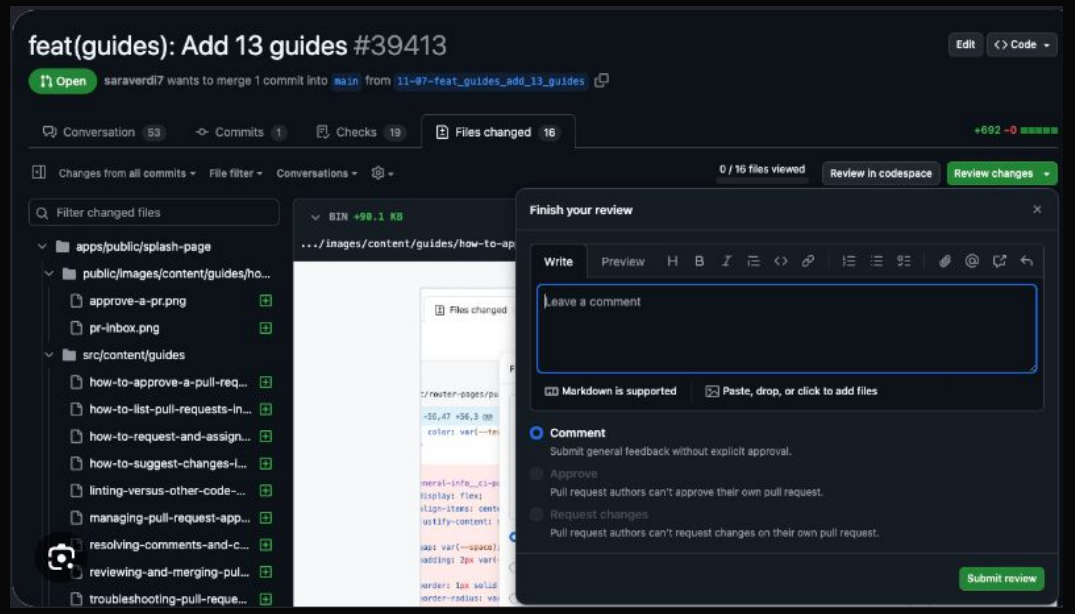
Resolver, compilar y probar

Sin conflictos de Git también hay que probar

En escenas y prefabs, evitar “todo mío / todo suyo” a ciegas.

Un PR listo para revisar

Destino: main.
Descripción: qué cambió y por qué.
Prueba: pasos concretos y resultado esperado.
Pendientes: visibles. Si falta trabajo, Draft.



Comentarios del revisor → **Correcciones** → **Commit y push** → **Mismo PR actualizado**

Otra persona revisa el diff y comprueba el comportamiento.

El PR se prueba antes de integrarlo

Abrir un PR pide una revisión. El merge incorpora los cambios a main.

CÓMO PROBAR LA RAMA DEL PR

1. Guardá tu trabajo pendiente y cerrá Unity antes de cambiar de rama.

2. Desktop: Current Branch → Pull Requests → elegí el PR. Fetch / Pull descarga su última versión.

3. Abrí Unity en esa rama. Verificá que compile y probá en Play lo indicado en el PR.

4. Revisá los archivos cambiados (diff) en GitHub y aprobá el PR o pedí correcciones.

ANTES: EL AUTOR ABRE EL PR

Prueba su rama en Unity, hace push y abre el PR con los pasos para probarla.

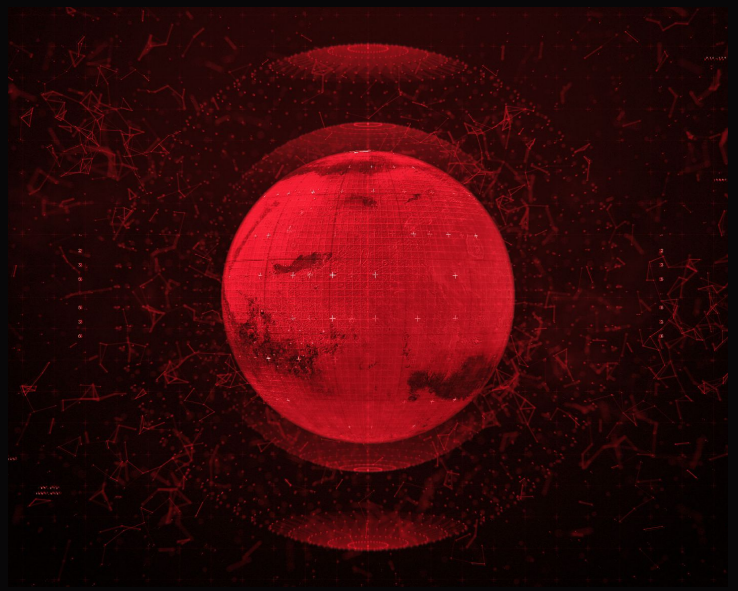
DESPUÉS: HACER MERGE

Pruebas correctas, revisión aprobada y sin conflictos. Hoy probamos manualmente; aún no hay checks automáticos. (Podría desarrollarlos si interesa)

¿Hubo correcciones? El autor hace commit y push; el revisor vuelve a probar. Si cambió main, actualizar la rama y repetir las pruebas afectadas.

Demo: recompensa de monedas

Estado inicial: cada clic suma 1.
Nuestra tarea: sumar 5 y agregar Reiniciar.
Otro cambio en main:
texto “Oro” y recompensa de 2.



Resultado acordado: texto “Oro”, suma de 5 y Reiniciar.

Después del merge: actualizar y probar

1. Descargar los cambios integrados

Tu rama en GitHub → **Merge del PR** → **main en GitHub** → **Pull** → **main en tu PC**

Guardar el trabajo y cerrar Unity → cambiar a main → Fetch / Pull.

2. Abrir Unity y comprobar el contador de oro

Inicio

Oro: 0 → Cobrar → **Oro: 5** → Cobrar → **Oro: 10** → Reiniciar → **Oro: 0**

Debe sumar de a 5, volver a 0 al reiniciar y conservar la etiqueta “Oro”.

Esta comprobación final en main complementa las pruebas realizadas antes del merge.

Tres preguntas para cerrar

Hice push. ¿Mi tarea ya está en main?
Hice pull de mi rama. ¿Ya tengo lo nuevo de main?
Resolví el conflicto. ¿Ya puedo integrar?

Tras integrar: borrar la rama. Próxima tarea: main actualizado y rama nueva.